

L23: Pointer Analysis

Software Analysis

IIT Guwahati · September 28, 2026

Introduction to Pointers (1/5)

No pointers

```
int x = 1;  
int y = x;  
assert(y == 1);
```

Introduction to Pointers (2/5)

No pointers

```
[x == 1] → int x = 1;  
[y == 1] → int y = x;  
          assert(y == 1);
```

A forward **must** analysis
over variables. **Proved.**

Introduction to Pointers (3/5)

No pointers

```
[x == 1] → int x = 1;  
[y == 1] → int y = x;  
          assert(y == 1);
```

A forward **must** analysis
over variables. **Proved.**

Same idea, with pointers

```
Circle *x = new Circle();  
x->radius = 1;  
int y = x->radius;  
assert(y == 1);
```

Introduction to Pointers (4/5)

No pointers

```
[x == 1] → int x = 1;  
[y == 1] → int y = x;  
          assert(y == 1);
```

A forward **must** analysis
over variables. **Proved.**

Same idea, with pointers

```
[x->radius == 1] → Circle *x = new Circle();  
[y == 1] → x->radius = 1;  
          int y = x->radius;  
          assert(y == 1);
```

Introduction to Pointers (5/5)

No pointers

```
[x == 1] → int x = 1;  
[y == 1] → int y = x;  
          assert(y == 1);
```

A forward **must** analysis
over variables. **Proved.**

Same idea, with pointers

```
Circle *x = new Circle();  
x->radius = 1;  
[x->radius == 1] → int y = x->radius;  
[y == 1] → assert(y == 1);
```

The facts are now about **expressions**,
not variables.

What a Pointer Holds (1/4)

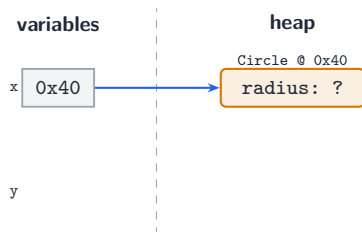
A Pointer Is an Address

```
Circle *x = new Circle();
```

```
x->radius = 1;
```

```
int y = x->radius;
```

x holds an **address**, not a circle.



What a Pointer Holds (2/4)

A Pointer Is an Address

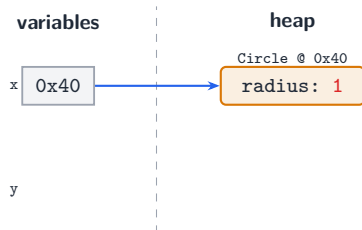
```
Circle *x = new Circle();
```

```
x->radius = 1;
```

```
int y = x->radius;
```

x holds an **address**, not a circle.

A **field write** follows the arrow, then fills the slot.



What a Pointer Holds (3/4)

A Pointer Is an Address

```
Circle *x = new Circle();
```

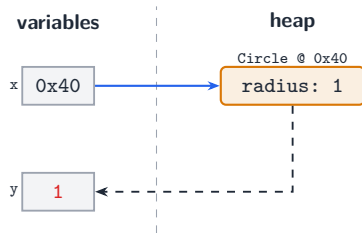
```
x->radius = 1;
```

```
int y = x->radius;
```

x holds an **address**, not a circle.

A **field write** follows the arrow, then fills the slot.

A **field read** follows the same arrow.



What a Pointer Holds (4/4)

A Pointer Is an Address

```
Circle *x = new Circle();
```

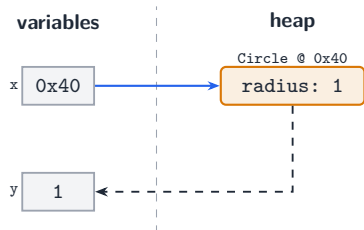
```
x->radius = 1;
```

```
int y = x->radius;
```

x holds an **address**, not a circle.

A **field write** follows the arrow, then fills the slot.

A **field read** follows the same arrow.

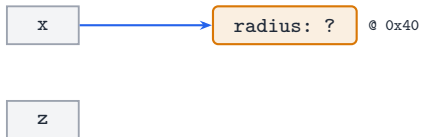


No aliases here: only x holds 0x40; y holds a copy of the *value*.

Pointer Aliasing (1/5)

When Two Expressions Name the Same Memory

```
Circle *x = new Circle();  
Circle *z = ???;  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```



Pointer Aliasing (2/5)

When Two Expressions Name the Same Memory

```
Circle *x = new Circle();
```

```
Circle *z = ???;
```

```
x->radius = 1;
```

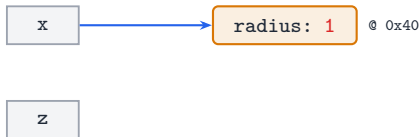
```
z->radius = 2;
```

```
int y = x->radius;
```

```
assert(y == 1);
```

The first write is easy:
x->radius is 1.

[x->radius == 1] →



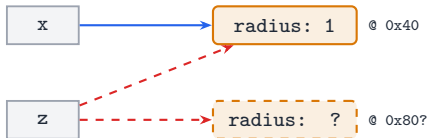
Pointer Aliasing (3/5)

When Two Expressions Name the Same Memory

```
Circle *x = new Circle();  
Circle *z = ???;  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

[x->radius == 1] →

[x->radius == ?] →



The first write is easy:
x->radius is 1.

The second: does writing
through z disturb
x->radius?

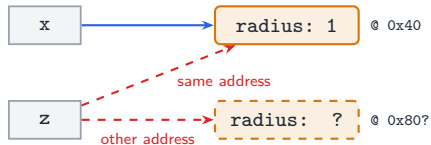
Pointer Aliasing (4/5)

When Two Expressions Name the Same Memory

```
Circle *x = new Circle();  
Circle *z = ???;  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

[x->radius == 1] →

[x->radius == ?] →



The first write is easy:
`x->radius` is 1.

The second: does writing
through `z` disturb
`x->radius`?

Only if `x` and `z` hold the
same address.

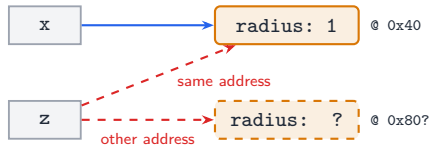
Pointer Aliasing (5/5)

When Two Expressions Name the Same Memory

```
Circle *x = new Circle();  
Circle *z = ???;  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

[x->radius == 1] →

[x->radius == ?] →



The first write is easy:
`x->radius` is 1.

The second: does writing
through `z` disturb
`x->radius`?

Only if `x` and `z` hold the
same address.

Such names are **aliases**.

Case 1: Different Circles (1/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

Case 1: Different Circles (2/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x != z] → x->radius = 1;  
           z->radius = 2;  
           int y = x->radius;  
           assert(y == 1);
```

Case 1: Different Circles (3/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x != z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

Case 1: Different Circles (4/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x != z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 1] → int y = x->radius;  
assert(y == 1);
```

The fact survives the second
write for exactly one reason:
the analysis knew

Case 1: Different Circles (5/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x != z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 1] → int y = x->radius;  
[y == 1] → assert(y == 1); OK
```

The fact survives the second write for exactly one reason: the analysis knew

Case 1: Different Circles (6/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x != z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 1] → int y = x->radius;  
[y == 1] → assert(y == 1); OK
```

The fact survives the second write for exactly one reason: the analysis knew

NOT x may-alias z

Proving facts of this shape is **may-alias analysis** — what we mean by **pointer analysis**.

Case 2: The Same Circle (1/6)

```
Circle *x = new Circle();  
Circle *z = x;  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

Case 2: The Same Circle (2/6)

```
Circle *x = new Circle();  
Circle *z = x;  
[x == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

Case 2: The Same Circle (3/6)

```
Circle *x = new Circle();  
Circle *z = x;  
[x == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 2] → int y = x->radius;  
[y == 2] → assert(y == 1); FAILS
```

Case 2: The Same Circle (4/6)

```
Circle *x = new Circle();  
Circle *z = x;  
[x == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 2] → int y = x->radius;  
[y == 2] → assert(y == 1); FAILS
```

The analysis is right and the *assertion* is wrong: *y* really is 2 here.

Case 2: The Same Circle (5/6)

```
Circle *x = new Circle();  
Circle *z = x;  
[x == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 2] → int y = x->radius;  
[y == 2] → assert(y == 1); FAILS
```

The analysis is right and the *assertion* is wrong: *y* really is 2 here.

This proof needed the other kind of fact:

x **must**-alias *z*

Case 2: The Same Circle (6/6)

```
Circle *x = new Circle();  
Circle *z = x;  
[x == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius == 2] → int y = x->radius;  
[y == 2] → assert(y == 1); FAILS
```

The analysis is right and the *assertion* is wrong: *y* really is 2 here.

This proof needed the other kind of fact:

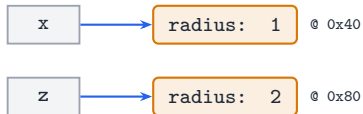
x **must**-alias *z*

Killing a fact requires knowing the write *definitely* hits it.

Where the Pointers Point (1/4)

The Two Cases as Memory

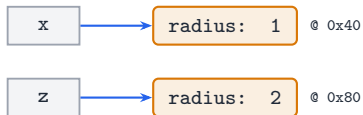
Case 1: `z = new Circle()`



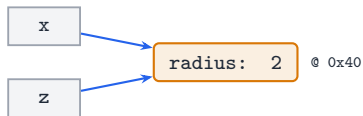
Where the Pointers Point (2/4)

The Two Cases as Memory

Case 1: `z = new Circle()`



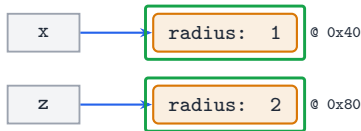
Case 2: `z = x`



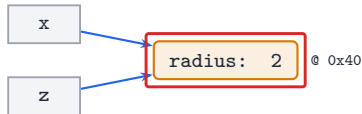
Where the Pointers Point (3/4)

The Two Cases as Memory

Case 1: `z = new Circle()`



Case 2: `z = x`

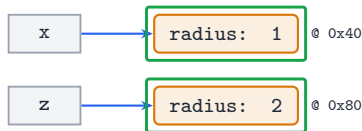


`x` and `z` **alias** when their arrows end on the **same object**.

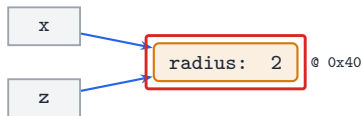
Where the Pointers Point (4/4)

The Two Cases as Memory

Case 1: `z = new Circle()`



Case 2: `z = x`



`x` and `z` **alias** when their arrows end on the **same object**.

Case 1: **never**. Case 2: **always**. Real programs can do *either* — per run.

Defining May- and Must-Alias (1/5)

```
Circle *x = new Circle(); // A
Circle *y = new Circle(); // B
Circle *z;
if (c) z = x; else z = y;
Circle *w = x;
```

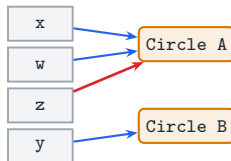
P →

Defining May- and Must-Alias (2/5)

```
Circle *x = new Circle(); // A
Circle *y = new Circle(); // B
Circle *z;
if (c) z = x; else z = y;
Circle *w = x;
```

P →

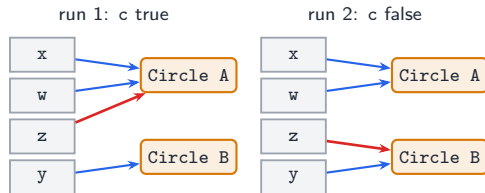
run 1: c true



Defining May- and Must-Alias (3/5)

```
Circle *x = new Circle(); // A
Circle *y = new Circle(); // B
Circle *z;
if (c) z = x; else z = y;
Circle *w = x;
```

P →

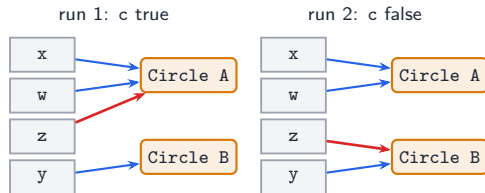


Defining May- and Must-Alias (4/5)

```
Circle *x = new Circle(); // A
Circle *y = new Circle(); // B
Circle *z;
if (c) z = x; else z = y;
Circle *w = x;
```

P →

May-alias at **P**: same object in *some* run.



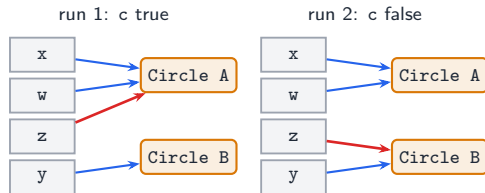
Defining May- and Must-Alias (5/5)

```
Circle *x = new Circle(); // A
Circle *y = new Circle(); // B
Circle *z;
if (c) z = x; else z = y;
Circle *w = x;
```

P →

May-alias at **P**: same object in *some* run.

Must-alias at **P**: same object in *every* run.



Examples (1/8)

Pair	run 1	run 2	At P
------	-------	-------	------

x, w			
------	--	--	--

x, z			
------	--	--	--

y, z			
------	--	--	--

x, y			
------	--	--	--

Examples (2/8)

Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z			
y, z			
x, y			

Examples (3/8)

Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z	same	differ	may , not must
y, z			
x, y			

Examples (4/8)

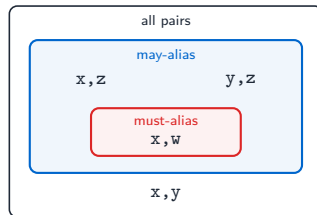
Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z	same	differ	may , not must
y, z	differ	same	may , not must
x, y			

Examples (5/8)

Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z	same	differ	may , not must
y, z	differ	same	may , not must
x, y	differ	differ	never : not may

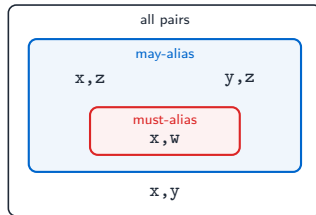
Examples (6/8)

Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z	same	differ	may , not must
y, z	differ	same	may , not must
x, y	differ	differ	never : not may



Examples (7/8)

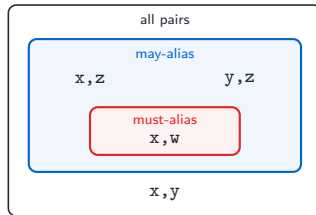
Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z	same	differ	may , not must
y, z	differ	same	may , not must
x, y	differ	differ	never : not may



Every must-alias pair is a may-alias pair: **must** $\subseteq$ **may**.

Examples (8/8)

Pair	run 1	run 2	At P
x, w	same	same	must (so also may)
x, z	same	differ	may , not must
y, z	differ	same	may , not must
x, y	differ	differ	never : not may



Every must-alias pair is a may-alias pair: **must** $\subseteq$ **may**.

An analysis sees no runs: it must *approximate* both sets.

May- and Must-Alias, Formally (1/5)

Quantify Over the Runs That Reach **P**

$$\textit{MayAlias}_P(p, q) \iff \exists \text{ run reaching } P: \text{ same address}$$

May- and Must-Alias, Formally (2/5)

Quantify Over the Runs That Reach **P**

$MayAlias_P(p, q) \iff \exists \text{ run reaching } P: \text{ same address}$

$MustAlias_P(p, q) \iff \forall \text{ runs reaching } P: \text{ same address}$

May- and Must-Alias, Formally (3/5)

Quantify Over the Runs That Reach **P**

$MayAlias_P(p, q) \iff \exists \text{ run reaching } P: \text{ same address}$

$MustAlias_P(p, q) \iff \forall \text{ runs reaching } P: \text{ same address}$

Answer	Meaning	Our example
Must-alias	definitely the same object	x, w
May-alias	possibly the same object	x, z
No-alias	definitely different objects	x, y

May- and Must-Alias, Formally (4/5)

Quantify Over the Runs That Reach **P**

$$\begin{aligned} \text{MayAlias}_P(p, q) &\iff \exists \text{ run reaching } P: \text{ same address} \\ \text{MustAlias}_P(p, q) &\iff \forall \text{ runs reaching } P: \text{ same address} \end{aligned}$$

Answer	Meaning	Our example
Must-alias	definitely the same object	x, w
May-alias	possibly the same object	x, z
No-alias	definitely different objects	x, y

Must $\Rightarrow$ *May* — provided some run reaches *P*.

May- and Must-Alias, Formally (5/5)

Quantify Over the Runs That Reach **P**

$$\begin{aligned} \text{MayAlias}_P(p, q) &\iff \exists \text{ run reaching } P: \text{ same address} \\ \text{MustAlias}_P(p, q) &\iff \forall \text{ runs reaching } P: \text{ same address} \end{aligned}$$

Answer	Meaning	Our example
Must-alias	definitely the same object	x, w
May-alias	possibly the same object	x, z
No-alias	definitely different objects	x, y

Must $\Rightarrow$ *May* — provided some run reaches *P*.

May $\not\Rightarrow$ *Must*: x, z is the witness.

Examples (1/5)

May, not must

```
int a, b;  
int *p = &a;  
int *q;  
if (c) q = &a; else q = &b;
```

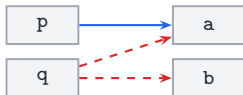
P →

Examples (2/5)

May, not must

```
int a, b;  
int *p = &a;  
int *q;  
if (c) q = &a; else q = &b;
```

P →

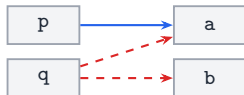


Examples (3/5)

May, not must

```
int a, b;  
int *p = &a;  
int *q;  
if (c) q = &a; else q = &b;
```

P →



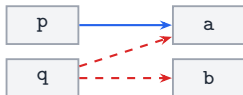
$pts(q) = \{a, b\}$: *some* run has
 $q \rightarrow a$.

Examples (4/5)

May, not must

```
int a, b;  
int *p = &a;  
int *q;  
if (c) q = &a; else q = &b;
```

$P \rightarrow$

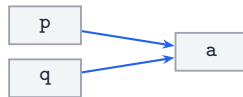


$pts(q) = \{a, b\}$: *some* run has $q \rightarrow a$.

Must

```
int a;  
int *p = &a;  
int *q = p;
```

$P \rightarrow$

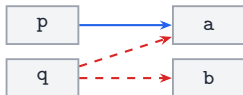


Examples (5/5)

May, not must

```
int a, b;  
int *p = &a;  
int *q;  
if (c) q = &a; else q = &b;
```

$P \rightarrow$

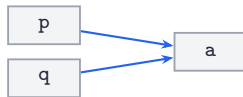


$pts(q) = \{a, b\}$: some run has $q \rightarrow a$.

Must

```
int a;  
int *p = &a;  
int *q = p;
```

$P \rightarrow$



Every run: both $\rightarrow a$. So also may.

Quiz: May, Must or Neither? (1/4)

```
Node *p = new Node(); // A
Node *q = new Node(); // B
Node *r;
if (c) r = p; else r = q;
Node *s = r;
Node *t = p;
if (d) t = q;
```

P →

At **P**, do these alias?

1. p, q

2. r, s

3. p, t

Quiz: May, Must or Neither? (2/4)

```
Node *p = new Node(); // A
Node *q = new Node(); // B
Node *r;
if (c) r = p; else r = q;
Node *s = r;
Node *t = p;
if (d) t = q;
```

P →

At **P**, do these alias?

1. p, q

Two sites, apart in every run.

2. r, s

3. p, t

Neither

Quiz: May, Must or Neither? (3/4)

```
Node *p = new Node(); // A
Node *q = new Node(); // B
Node *r;
if (c) r = p; else r = q;
Node *s = r;
Node *t = p;
if (d) t = q;
```

P →

At **P**, do these alias?

1. p, q

Neither

Two sites, apart in every run.

2. r, s

Must

s copies r, whatever c is.

3. p, t

Quiz: May, Must or Neither? (4/4)

```
Node *p = new Node(); // A
Node *q = new Node(); // B
Node *r;
if (c) r = p; else r = q;
Node *s = r;
Node *t = p;
if (d) t = q;
```

P →

At **P**, do these alias?

1. p, q

Neither

Two sites, apart in every run.

2. r, s

Must

s copies r, whatever c is.

3. p, t

May

Same if d is false, not if true.

May versus Must in Practice (1/5)

	May-alias	Must-alias
Proves	x, z never alias	x, z always alias
Used to		
Update		
Machinery		
In practice		

May versus Must in Practice (2/5)

	May-alias	Must-alias
Proves	x, z never alias	x, z always alias
Used to	<i>keep</i> a fact across a write	<i>kill</i> a fact at a write
Update		
Machinery		
In practice		

May versus Must in Practice (3/5)

	May-alias	Must-alias
Proves	x, z never alias	x, z always alias
Used to	<i>keep</i> a fact across a write	<i>kill</i> a fact at a write
Update	weak — only adds	strong — may remove
Machinery		
In practice		

May versus Must in Practice (4/5)

	May-alias	Must-alias
Proves	x, z never alias	x, z always alias
Used to	<i>keep</i> a fact across a write	<i>kill</i> a fact at a write
Update	weak — only adds	strong — may remove
Machinery	modest	far heavier
In practice	used everywhere	rare

May versus Must in Practice (5/5)

	May-alias	Must-alias
Proves	x, z never alias	x, z always alias
Used to	<i>keep</i> a fact across a write	<i>kill</i> a fact at a write
Update	weak — only adds	strong — may remove
Machinery	modest	far heavier
In practice	used everywhere	rare

They are duals, but they are not equally affordable. For the rest of this lecture, **pointer analysis** means **may-alias analysis**.

Clients of Alias Analysis (1/5)

Redundant Load Elimination

```
int f(int *p, int *q) {  
    *p = 10;  
    *q = 20;  
    return *p;  
}
```

Relation of p, q	Result of return *p
------------------	---------------------

Can the compiler replace
return *p with a constant?

Clients of Alias Analysis (2/5)

Redundant Load Elimination

```
int f(int *p, int *q) {  
    *p = 10;  
    *q = 20;  
    return *p;  
}
```

Relation of p, q

No-alias

Result of return *p

return 10

Can the compiler replace
return *p with a constant?

Clients of Alias Analysis (3/5)

Redundant Load Elimination

```
int f(int *p, int *q) {  
    *p = 10;  
    *q = 20;  
    return *p;  
}
```

Relation of p, q

No-alias

Must-alias

Result of return *p

return 10

return 20

Can the compiler replace
return *p with a constant?

Clients of Alias Analysis (4/5)

Redundant Load Elimination

```
int f(int *p, int *q) {  
    *p = 10;  
    *q = 20;  
    return *p;  
}
```

Relation of p, q

No-alias

Must-alias

May-alias

Result of return *p

return 10

return 20

reload *p — no shortcut

Can the compiler replace
return *p with a constant?

Clients of Alias Analysis (5/5)

Redundant Load Elimination

```
int f(int *p, int *q) {  
    *p = 10;  
    *q = 20;  
    return *p;  
}
```

Can the compiler replace
return *p with a constant?

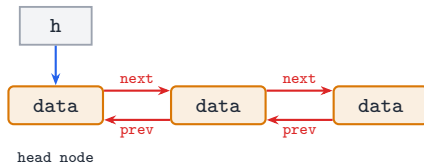
Relation of p, q	Result of return *p
No-alias	return 10
Must-alias	return 20
May-alias	reload *p — no shortcut

Other clients: **constant propagation**, **dead-store elimination**, **race detection**, **taint analysis**, and function-pointer **call graphs**.

Why Pointers Are Hard (1/6)

Infinitely Many Names for One Field

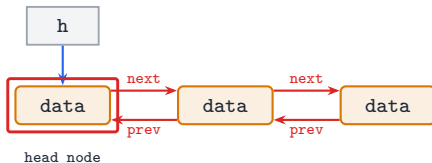
```
typedef struct Node {  
    int data;  
    struct Node *next, *prev;  
} Node;  
Node *h = NULL;  
while (more()) {  
    Node *v = malloc(sizeof(Node));  
    v->next = h;  
    if (h) h->prev = v;  
    h = v;  
}
```



Why Pointers Are Hard (2/6)

Infinitely Many Names for One Field

```
typedef struct Node {  
    int data;  
    struct Node *next, *prev;  
} Node;  
Node *h = NULL;  
while (more()) {  
    Node *v = malloc(sizeof(Node));  
    v->next = h;  
    if (h) h->prev = v;  
    h = v;  
}
```

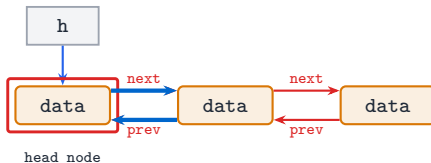


Names for the head node's data:
h->data

Why Pointers Are Hard (3/6)

Infinitely Many Names for One Field

```
typedef struct Node {  
    int data;  
    struct Node *next, *prev;  
} Node;  
Node *h = NULL;  
while (more()) {  
    Node *v = malloc(sizeof(Node));  
    v->next = h;  
    if (h) h->prev = v;  
    h = v;  
}
```



Names for the head node's data:

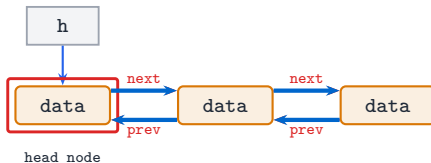
`h->data`

`h->next->prev->data`

Why Pointers Are Hard (4/6)

Infinitely Many Names for One Field

```
typedef struct Node {  
    int data;  
    struct Node *next, *prev;  
} Node;  
Node *h = NULL;  
while (more()) {  
    Node *v = malloc(sizeof(Node));  
    v->next = h;  
    if (h) h->prev = v;  
    h = v;  
}
```



Names for the head node's data:

`h->data`

`h->next->prev->data`

`h->next->next->prev->prev->data`

`h->next->prev->next->prev->data`

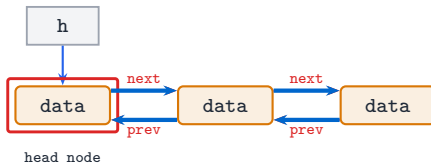
...

Why Pointers Are Hard (5/6)

Infinitely Many Names for One Field

```
typedef struct Node {  
    int data;  
    struct Node *next, *prev;  
} Node;  
Node *h = NULL;  
while (more()) {  
    Node *v = malloc(sizeof(Node));  
    v->next = h;  
    if (h) h->prev = v;  
    h = v;  
}
```

Cycles $\Rightarrow$ **infinitely many**
names.



Names for the head node's data:

`h->data`

`h->next->prev->data`

`h->next->next->prev->prev->data`

`h->next->prev->next->prev->data`

...

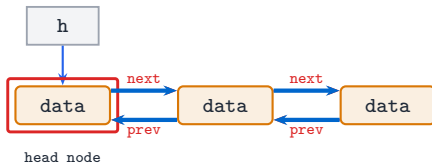
Why Pointers Are Hard (6/6)

Infinitely Many Names for One Field

```
typedef struct Node {  
    int data;  
    struct Node *next, *prev;  
} Node;  
Node *h = NULL;  
while (more()) {  
    Node *v = malloc(sizeof(Node));  
    v->next = h;  
    if (h) h->prev = v;  
    h = v;  
}
```

Cycles $\Rightarrow$ **infinitely many** names.

So: **abstract memory itself**.



Names for the head node's data:

`h->data`

`h->next->prev->data`

`h->next->next->prev->prev->data`

`h->next->prev->next->prev->data`

...

Aliasing Is Undecidable (1/5)

Why No Analysis Can Answer Exactly

```
Circle *x = new Circle();  
Circle *z = new Circle();  
P();    // any code at all  
z = x;  
  
// can x and z ever alias?
```

Aliasing Is Undecidable (2/5)

Why No Analysis Can Answer Exactly

```
Circle *x = new Circle();  
Circle *z = new Circle();  
P();    // any code at all  
z = x;  
// can x and z ever alias?
```

Two news give two different addresses.
Only $z = x$ can make them equal.

Aliasing Is Undecidable (3/5)

Why No Analysis Can Answer Exactly

```
Circle *x = new Circle();  
Circle *z = new Circle();  
P();    // any code at all  
z = x;  
// can x and z ever alias?
```

Two news give two different addresses.
Only $z = x$ can make them equal.
And $z = x$ runs on some execution *if and only if* $P()$ halts.

Aliasing Is Undecidable (4/5)

Why No Analysis Can Answer Exactly

```
Circle *x = new Circle();  
Circle *z = new Circle();  
P();    // any code at all  
z = x;  
// can x and z ever alias?
```

Two news give two different addresses.
Only $z = x$ can make them equal.

And $z = x$ runs on some execution *if and only if* $P()$ halts.

So an exact MayAlias would decide whether an arbitrary P halts — the **halting problem**.

Aliasing Is Undecidable (5/5)

Why No Analysis Can Answer Exactly

```
Circle *x = new Circle();  
Circle *z = new Circle();  
P();    // any code at all  
z = x;  
// can x and z ever alias?
```

Two news give two different addresses.
Only $z = x$ can make them equal.

And $z = x$ runs on some execution *if and only if* $P()$ halts.

So an exact MayAlias would decide whether an arbitrary P halts — the **halting problem**.

So every pointer analysis must **approximate**.

Undecidable, So We Approximate (1/6)

Deciding whether two pointers alias is **undecidable**.

Undecidable, So We Approximate (2/6)

Deciding whether two pointers alias is **undecidable**.

- We may give up **soundness**, **completeness** or **termination**.

Undecidable, So We Approximate (3/6)

Deciding whether two pointers alias is **undecidable**.

- We may give up **soundness**, **completeness** or **termination**.
- As before, we give up **completeness**.

Undecidable, So We Approximate (4/6)

Deciding whether two pointers alias is **undecidable**.

- We may give up **soundness**, **completeness** or **termination**.
- As before, we give up **completeness**.
- So `x MayAlias z` returns **NO** only when it is *certain* they never alias.

Undecidable, So We Approximate (5/6)

Deciding whether two pointers alias is **undecidable**.

- We may give up **soundness**, **completeness** or **termination**.
- As before, we give up **completeness**.
- So `x MayAlias z` returns **NO** only when it is *certain* they never alias.
- It returns **YES** whenever it cannot tell — even if they in fact never alias.

Undecidable, So We Approximate (6/6)

Deciding whether two pointers alias is **undecidable**.

- We may give up **soundness**, **completeness** or **termination**.
- As before, we give up **completeness**.
- So `x MayAlias z` returns **NO** only when it is *certain* they never alias.
- It returns **YES** whenever it cannot tell — even if they in fact never alias.

A wrong **YES** is a **false positive**. A wrong **NO** would be unsound — never allowed.

What a False Positive Costs (1/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
x->radius = 1;  
z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

What a False Positive Costs (2/6)

```
[x may == z] → Circle *x = new Circle();  
                Circle *z = new Circle();  
                x->radius = 1;  
                z->radius = 2;  
                int y = x->radius;  
                assert(y == 1);
```

Say the pointer analysis
answers **YES**.

What a False Positive Costs (3/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x may == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
int y = x->radius;  
assert(y == 1);
```

Say the pointer analysis
answers **YES**.

What a False Positive Costs (4/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x may == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius ∈ {1,2}] → int y = x->radius;  
assert(y == 1);
```

Say the pointer analysis
answers **YES**.

Data-flow must then keep
both values.

What a False Positive Costs (5/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x may == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius ∈ {1,2}] → int y = x->radius;  
[y ∈ {1,2}] → assert(y == 1); UNPROVED
```

Say the pointer analysis answers **YES**.

Data-flow must then keep *both* values.

So the assertion is unproved — though it always holds.

What a False Positive Costs (6/6)

```
Circle *x = new Circle();  
Circle *z = new Circle();  
[x may == z] → x->radius = 1;  
[x->radius == 1] → z->radius = 2;  
[x->radius ∈ {1,2}] → int y = x->radius;  
[y ∈ {1,2}] → assert(y == 1); UNPROVED
```

Say the pointer analysis answers **YES**.

Data-flow must then keep *both* values.

So the assertion is unproved — though it always holds.

One imprecise answer, one **spurious warning**.

Approximation (1/4)

Every sound pointer analysis approximates. They differ in **how**, along two axes.

Approximation (2/4)

Every sound pointer analysis approximates. They differ in **how**, along two axes.

Abstract the heap

A program can create any number of objects. We group them into a few *abstract objects*.

This is the **heap abstraction**.

Approximation (3/4)

Every sound pointer analysis approximates. They differ in **how**, along two axes.

Abstract the heap

A program can create any number of objects. We group them into a few *abstract objects*.

This is the **heap abstraction**.

Abstract the control flow

We keep one result for the whole program, not one per program point.

This is the **flow abstraction**.

Approximation (4/4)

Every sound pointer analysis approximates. They differ in **how**, along two axes.

Abstract the heap

A program can create any number of objects. We group them into a few *abstract objects*.

This is the **heap abstraction**.

Abstract the control flow

We keep one result for the whole program, not one per program point.

This is the **flow abstraction**.

Data-flow analysis already abstracted branch conditions away. Pointer analysis goes further and discards *order* entirely.